

Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3084 – Data Science

Semestre II – 2026

Proyecto 2: Resultados

Reconocimiento de deletreo manual (fingerspelling) de la Lengua de Señas Americana a partir de landmarks de MediaPipe

Reto 15: Google – American Sign Language Fingerspelling Recognition

Integrante	**Carné**
Diego José López Campos	23242
Luis González	23353
Diego Rosales	23258

Guatemala, 12 de octubre de 2026

Índice

1. Introducción 4
2. Objetivos 5
2.1 Objetivo general 5
2.2 Objetivos específicos 5
3. Marco teórico 6
3.1 El deletreo manual en ASL 6
3.2 Reconocimiento automático de lenguas de señas 6
3.3 De la imagen a los puntos clave: estimación de pose y MediaPipe 6
3.4 Modelos para secuencias 7
3.5 Clasificación temporal conexionista (CTC) 7
3.6 Métrica de evaluación 8
4. Metodología 9
4.1 Fuente y estructura 9
4.2 Conjuntos de entrenamiento, validación y prueba 9
4.3 Preprocesamiento 9
4.4 Almacenamiento 10
4.5 Herramientas y cómputo 10
4.6 Selección de los algoritmos 11
4.7 Configuración común 12
4.8 Ajuste de hiperparámetros 12
4.9 Evaluación 12
5. Resultados y análisis 14
Conjunto de datos, limpieza y análisis exploratorio 14
5.1 El corpus de frases 14
5.2 Estructura y calidad de la señal 15
5.3 Qué estrategia de relleno usar 16
5.4 Artefactos de seguimiento 16
5.5 Dimensión temporal 17
5.6 Resultado de la preparación sobre el corpus completo 17
5.7 Limitaciones 18
Modelos 18
5.8 Ajuste de hiperparámetros 18
5.9 Entrenamiento final 19

5.10 Comparación en el conjunto de prueba 20
5.11 Análisis de errores 20
5.12 Aplicación interactiva 22
6. Conclusiones 29
6.1 Cumplimiento de los objetivos 29
6.2 Efectividad de los algoritmos 29
6.3 Limitaciones y trabajo futuro 30
7. Referencias 31
Anexo: código y artefactos 34

1. Introducción

La Organización Mundial de la Salud (2021) estima que más de 430 millones de personas necesitan rehabilitación por pérdida de audición y que la cifra superará los 700 millones en 2050. Para buena parte de la comunidad sorda, la lengua materna es una lengua de señas, y la tecnología de uso diario no la entiende: los asistentes de voz, los buscadores y los teclados predictivos están pensados para quien habla o escribe con soltura una lengua oral. Bragg et al. (2019) señalan que el reconocimiento de lenguas de señas sigue lejos de la madurez del reconocimiento de voz, en parte por la escasez de datos y en parte porque el problema combina visión por computadora, lingüística y accesibilidad.

Dentro de la Lengua de Señas Americana (ASL), el deletreo manual o *fingerspelling* tiene una función propia: representa palabras letra por letra con configuraciones de una sola mano. Se usa para nombres propios, direcciones, números de teléfono, correos, direcciones web y términos que no tienen una seña establecida. Padden y Gunsauls (2003) documentan que ocupa entre el 12 % y el 35 % del discurso en ASL. Es, además, justo el tipo de contenido que una persona necesita introducir en un teléfono, así que un sistema que no lo resuelva bien no sirve para la comunicación cotidiana.

El problema es difícil por tres razones. Primero, las letras no se producen aisladas: la forma de la mano en una letra depende de la anterior y de la siguiente (coarticulación), y los firmantes fluidos deletrean varias letras por segundo. Segundo, los datos no vienen segmentados: se conoce la frase completa de cada video, pero no en qué instante ocurre cada letra. Tercero, hay mucha variación entre firmantes en velocidad, lateralidad y estilo (Shi et al., 2018, 2019).

Este proyecto trabaja con el conjunto de datos del reto de Kaggle *Google – American Sign Language Fingerspelling Recognition* (Chow et al., 2023), recolectado con teléfonos por firmantes sordos y descrito por Georg et al. (2024). Los videos no se distribuyen: cada cuadro llega ya convertido en 543 puntos del cuerpo estimados con MediaPipe (Lugaresi et al., 2019). La tarea es convertir esa secuencia de coordenadas en la frase deletreada.

En la primera fase del proyecto se hizo el análisis exploratorio, que definió cómo limpiar los datos. En esta segunda fase se construye el sistema completo: una función de preprocesamiento única, tres modelos de reconocimiento de secuencias entrenados con la pérdida CTC, su comparación por distancia de edición, y una aplicación que recibe una secuencia cruda y muestra la transcripción de cada modelo.

2. Objetivos

2.1 Objetivo general

Desarrollar y evaluar un sistema que transcriba a texto secuencias de *fingerspelling* de ASL representadas como landmarks de MediaPipe, comparando tres arquitecturas de reconocimiento de secuencias entrenadas con CTC y poniendo el resultado al alcance de un usuario mediante una aplicación interactiva.

2.2 Objetivos específicos

1. **Implementar un preprocesamiento único y reproducible** que aplique las decisiones del análisis exploratorio (selección de canales, mano activa, invalidación de artefactos, recorte, interpolación limitada, normalización y máscara de validez) y que produzca exactamente las mismas características en el entrenamiento y en la aplicación.
2. **Construir los conjuntos de entrenamiento, validación y prueba separando firmantes**, de modo que ningún participante aparezca en más de un conjunto y la evaluación mida el desempeño con personas que el modelo no ha visto.
3. **Entrenar tres modelos con pérdida CTC** (una red recurrente bidireccional, una red convolucional temporal y un codificador basado en atención) sobre el corpus completo de 68 archivos.
4. **Comparar los modelos** con la distancia de edición normalizada como métrica principal y, como métricas secundarias, la tasa de error por carácter, la proporción de frases exactas, el error por tipo de carácter, el tiempo de inferencia y el número de parámetros.
5. **Relacionar los errores de los modelos con los hallazgos del análisis exploratorio**, en particular con el desbalance entre tipos de carácter y con la calidad de la señal de la mano.
6. **Entregar una aplicación** que reciba una secuencia cruda en el formato original, la preprocese sin intervención del usuario y presente la predicción de uno o varios modelos junto con su rendimiento.

3. Marco teórico

3.1 El deletreo manual en ASL

El alfabeto manual de ASL asigna una configuración de la mano a cada letra del alfabeto latino. Padden y Gunsauls (2003) describen cómo este sistema, tomado de la escritura, se integró a la lengua hasta volverse un recurso habitual y no un simple auxiliar. Para el reconocimiento automático importan tres propiedades. Es unimanual, así que una mano concentra casi toda la información. Es rápido y continuo, por lo que las configuraciones se funden unas con otras. Y su vocabulario es abierto: lo que se deletrea suele ser precisamente lo que no está en un diccionario, como nombres, números o direcciones.

3.2 Reconocimiento automático de lenguas de señas

El reconocimiento de señas pasó de sistemas estadísticos con características diseñadas a mano a redes neuronales profundas. Koller et al. (2015) muestran el enfoque estadístico para reconocimiento continuo con varios firmantes y vocabulario grande, y hacen explícito el problema que sigue vigente: la variación entre personas. La revisión de Rastgoo et al. (2021) ordena los trabajos con aprendizaje profundo según la entrada que usan (imagen, profundidad o esqueleto) y el tipo de tarea (señas aisladas o continuas). Camgoz et al. (2018) plantearon la traducción de señas como un problema de secuencia a secuencia, y Camgoz et al. (2020) lo resolvieron con Transformers entrenando de forma conjunta el reconocimiento y la traducción con una pérdida CTC intermedia.

Para *fingerspelling* en condiciones reales, Shi et al. (2018) recopilaron videos de ASL de internet y mostraron que el reconocimiento "en el mundo real" es mucho más difícil que en estudio. Shi et al. (2019) mejoraron el resultado con un mecanismo de atención visual que se acerca iterativamente a la mano, sin depender de un detector de manos aparte. Shi et al. (2021) abordaron después el problema previo de detectar en qué tramos de un video se está deletreando. El conjunto que usa este proyecto es de otra escala: Georg et al. (2024) reportan más de tres millones de caracteres deletreados por firmantes sordos frente a la cámara de un teléfono.

3.3 De la imagen a los puntos clave: estimación de pose y MediaPipe

Un video es una entrada muy costosa: cada cuadro tiene cientos de miles de pixeles y casi todos son fondo. La estimación de pose reduce cada cuadro a las coordenadas de unas cuantas articulaciones. OpenPose (Cao et al., 2021) mostró que es posible estimar en tiempo real los puntos del cuerpo, las manos y la cara de varias personas a partir de una imagen común. MediaPipe (Lugaresi et al., 2019) es un marco de trabajo para armar este tipo de flujos de percepción y ejecutarlos en dispositivos móviles. Su modelo de manos (Zhang et al., 2020) primero detecta la palma y luego regresa 21 puntos tridimensionales de la mano.

Los datos del reto se generaron con la solución *Holistic* de MediaPipe, que entrega 543 puntos por cuadro: 468 de la cara, 33 del cuerpo y 21 de cada mano. Cada punto tiene coordenadas x e y normalizadas al ancho y alto de la imagen, y una z de profundidad relativa. Trabajar con puntos tiene dos ventajas: la

entrada es pequeña y no contiene la imagen de la persona. Tiene también un costo, que el análisis exploratorio midió: cuando el detector no encuentra una mano, ese cuadro llega vacío, y la calidad de todo lo demás depende de la calidad de esa estimación.

3.4 Modelos para secuencias

Las redes neuronales profundas aprenden representaciones en capas a partir de los datos, sin características diseñadas a mano (LeCun et al., 2015). Para secuencias hay tres familias que este proyecto compara.

Redes recurrentes. Procesan la secuencia paso a paso y mantienen un estado que resume lo visto. Las redes recurrentes simples olvidan con facilidad las dependencias largas; la LSTM (Hochreiter y Schmidhuber, 1997) lo resuelve con compuertas que controlan qué se guarda y qué se descarta. La GRU (Cho et al., 2014) es una variante con menos compuertas y menos parámetros. Una red bidireccional (Schuster y Paliwal, 1997) recorre la secuencia en ambos sentidos, de modo que la salida en cada instante conoce el contexto anterior y el posterior. Eso encaja con la coarticulación: la forma de una letra depende de las vecinas. Graves et al. (2013) mostraron que las redes recurrentes profundas y bidireccionales entrenadas de extremo a extremo son competitivas en reconocimiento de voz, un problema con la misma estructura que este.

Redes convolucionales temporales. Aplican convoluciones de una dimensión a lo largo del tiempo. Lea et al. (2017) las propusieron para segmentar acciones en video y mostraron que capturan patrones de larga duración con un entrenamiento más rápido que el de las recurrentes. Bai et al. (2018) compararon de forma sistemática convoluciones con dilatación contra redes recurrentes y encontraron que las primeras igualan o superan a las segundas en varias tareas de secuencias. Su ventaja práctica es que todos los instantes se calculan en paralelo.

Atención y Transformers. El Transformer (Vaswani et al., 2017) elimina la recurrencia: cada posición de la secuencia consulta directamente a todas las demás mediante autoatención. Captura dependencias globales, pero no tiene de por sí una noción de vecindad local. El Conformer (Gulati et al., 2020) combina autoatención con convoluciones para modelar a la vez lo global y lo local, y se volvió una referencia en reconocimiento de voz. Squeezeformer (Kim et al., 2022) reorganiza esa arquitectura para reducir su costo.

3.5 Clasificación temporal conexionista (CTC)

Los datos no dicen en qué cuadro ocurre cada letra. La clasificación temporal conexionista (Graves et al., 2006) permite entrenar sin esa alineación. El modelo emite, en cada uno de los *T* cuadros, una distribución sobre el vocabulario más un símbolo especial llamado *blank*. Una secuencia de *T* símbolos es un camino π, y la función *B* lo convierte en texto fusionando los símbolos repetidos consecutivos y eliminando después los *blank*. Muchos caminos distintos producen la misma frase **y**, y su probabilidad es la suma sobre todos ellos:

$$p(\mathbf{y} \mid \mathbf{x}) = \Sigma_{\pi \in B^{-1}(y)} \prod_{t=1}^{T} p(\pi_t \mid \mathbf{x})$$

La pérdida es el logaritmo negativo de esa probabilidad, y se calcula de forma eficiente con programación dinámica. De la definición se sigue una restricción que se usó al preparar los datos: la entrada debe tener al menos tantos cuadros como caracteres tiene la frase, más uno por cada par de caracteres iguales consecutivos, porque entre ellos hace falta un *blank*. Para predecir se usa la decodificación voraz: se toma el símbolo más probable de cada cuadro y se aplica *B*.

3.6 Métrica de evaluación

La exactitud por clase no aplica cuando la salida es una cadena de longitud variable. La distancia de Levenshtein (1966) cuenta el número mínimo de inserciones, eliminaciones y sustituciones de un carácter que convierten una cadena en otra. La competencia usa la distancia de edición normalizada por la longitud de la frase real (Chow et al., 2023), y esa es la métrica principal del proyecto. La tasa de error por carácter (CER) divide las ediciones totales entre los caracteres totales del conjunto.

4. Metodología

4.1 Fuente y estructura

Los datos provienen del reto de Kaggle (Chow et al., 2023). Tienen dos niveles unidos por `sequence_id`:

Archivo	Una fila es	Contenido
`train.csv`	una secuencia	ruta, `file_id`, `sequence_id`, `participant_id`, `phrase`
`train_landmarks/*.parquet` (68 archivos)	un cuadro	`sequence_id`, `frame` y 1,629 coordenadas
`character_to_prediction_index.json`	un carácter	índice de cada carácter del vocabulario

El corpus tiene 67,208 secuencias de 94 participantes y ocupa cerca de 190 GB.

4.2 Conjuntos de entrenamiento, validación y prueba

La partición se hizo **por participante**, no por secuencia. El análisis exploratorio encontró que el 38 % de la varianza en la velocidad de deletreo se explica por el firmante; si la misma persona estuviera en entrenamiento y en prueba, el modelo podría aprender su estilo y la evaluación sería optimista. Los 94 participantes se barajaron con una semilla fija (42) y se repartieron 80 % / 10 % / 10 %:

Conjunto	Participantes	Secuencias	% de secuencias	Longitud media de la frase
Entrenamiento	75	54,377	80.9	17.78
Validación	9	6,894	10.3	17.84
Prueba	10	5,937	8.8	18.00

La partición se define sobre `train.csv`, así que es la misma sin importar cuántos archivos se procesen. El conjunto de validación se usa para elegir el mejor punto de entrenamiento y para ajustar hiperparámetros. El de prueba se reserva para una sola evaluación final.

4.3 Preprocesamiento

Todo el preprocesamiento vive en una función, `preprocess_sequence` (`src/preprocessing.py`), que recibe una secuencia cruda y devuelve una matriz de características y una máscara de validez. El entrenamiento y la aplicación llaman a la misma función. Los pasos, en orden, son:

1. **Selección de canales.** Se leen las dos manos (126 coordenadas) y siete puntos del cuerpo en *x* e *y*: nariz, hombros, codos y muñecas. La malla facial se descarta: es el 86.2 % de las columnas y no participa en el deletreo.
2. **Mano activa.** Se elige la mano con mayor proporción de cuadros detectados. Si es la izquierda, la secuencia se refleja horizontalmente para que todas parezcan diestras.

3. **Invalidación de artefactos.** Se calcula el desplazamiento del centroide de la mano entre cuadros consecutivos. Un cuadro se invalida solo si es atípico por dos criterios a la vez: la puntuación *z* robusta basada en la desviación absoluta mediana supera 3.5 (Iglewicz y Hoaglin, 1993) y el desplazamiento supera 0.15 unidades normalizadas por cuadro, un salto físicamente imposible para una mano.
4. **Recorte.** Se eliminan los tramos sin mano al inicio y al final.
5. **Interpolación limitada.** Los huecos internos de hasta 5 cuadros se rellenan por interpolación lineal. Los más largos se dejan vacíos.
6. **Normalización.** La mano se centra en la muñeca y se divide por la distancia entre la muñeca y la base del dedo medio, lo que elimina la posición y el tamaño aparente. La posición de la muñeca y los puntos del cuerpo se expresan respecto al punto medio de los hombros y se dividen por el ancho de hombros.
7. **Máscara de validez.** Se marca cada cuadro como válido o no. Los no válidos se rellenan con cero solo para que la matriz sea numérica; el modelo los ignora.

El resultado son **80 características por cuadro**: la mano activa normalizada (63), la posición de la muñeca respecto al cuerpo (2), los siete puntos del cuerpo (14) y un indicador de si la otra mano aparece (1).

Dos decisiones se dejaron abiertas a propósito. El umbral de descarte por cobertura (el análisis exploratorio proponía exigir 50 % de cuadros válidos) no se aplica al preparar los datos: se guarda la cobertura de cada secuencia y el umbral se decide al entrenar. Tampoco se descartan aquí las secuencias con menos cuadros que los que exige CTC; se marcan, y el cargador de datos las excluye.

4.4 Almacenamiento

Las secuencias tienen longitudes distintas. Se guardan concatenadas en un solo arreglo en `float16`, con un índice que registra dónde empieza y cuánto mide cada una. El corpus procesado ocupa 1.6 GB frente a los 190 GB originales.

4.5 Herramientas y cómputo

Componente	Herramienta
Lenguaje	Python 3
Lectura por columnas de los Parquet	PyArrow
Cálculo numérico y tablas	NumPy (Harris et al., 2020), pandas, SciPy
Modelos y entrenamiento	PyTorch (Paszke et al., 2019)
Gráficas del informe	Matplotlib, seaborn
Aplicación	Streamlit, Plotly
Control de versiones	Git y GitHub

El preprocesamiento del corpus completo se ejecutó en un notebook de Kaggle, donde la competencia ya está montada como fuente de datos y no hace falta descargar nada. El notebook clona el repositorio para usar el mismo código de `src/`. Como solo se leen 141 de las 1,631 columnas de cada archivo, las 67,208 secuencias (10.6 millones de cuadros) se procesaron en unos 9 minutos de CPU. Los tres modelos se entrenaron en el mismo notebook de Kaggle con una GPU NVIDIA T4; el tiempo de entrenamiento de cada uno se reporta en la sección 5.9.

4.6 Selección de los algoritmos

La revisión bibliográfica y el análisis exploratorio llevan a la misma formulación: el problema es de **reconocimiento de secuencias**, no de clasificación. Cada muestra es una serie de unos 160 cuadros de landmarks que debe transcribirse a una frase de longitud variable, y los datos no indican en qué cuadro empieza cada letra. Por eso los tres modelos comparten la salida y la función de pérdida. En cada cuadro emiten una distribución sobre los 59 caracteres del vocabulario oficial más el símbolo *blank*, y se entrenan con CTC (Graves et al., 2006). Lo único que cambia entre ellos es el codificador temporal, que es justo lo que la literatura discute para este tipo de problema:

Modelo	Codificador	Justificación
M1. BiGRU + CTC	Dos capas GRU bidireccionales	Las redes recurrentes bidireccionales con CTC son la línea base clásica del reconocimiento de voz de extremo a extremo (Graves et al., 2013) y de *fingerspelling* en video (Shi et al., 2018). La GRU (Cho et al., 2014) tiene menos parámetros que la LSTM (Hochreiter y Schmidhuber, 1997). La bidireccionalidad (Schuster y Paliwal, 1997) permite que cada cuadro vea la letra anterior y la siguiente, lo que conviene a la coarticulación observada en el EDA.
M2. TCN + CTC	Convoluciones temporales dilatadas con conexiones residuales	Lea et al. (2017) y Bai et al. (2018) muestran que las convoluciones temporales igualan a las recurrentes en tareas de secuencias, con cálculo paralelo. El EDA mostró que la información está en transiciones locales entre configuraciones de mano, que una ventana convolucional captura directamente.
M3. Transformer + CTC	Encoder Transformer con codificación posicional sinusoidal	La autoatención (Vaswani et al., 2017) relaciona cualquier par de cuadros. Es la base de los Sign Language Transformers (Camgoz et al., 2020) y de las arquitecturas Conformer y Squeezeformer (Gulati et al., 2020; Kim et al., 2022) que dominaron la competencia.

Se descartaron explícitamente dos alternativas:

- **Clasificadores de una letra por cuadro o por ventana** (por ejemplo, *random forest* o SVM sobre características agregadas): requieren saber dónde está cada letra, información que el conjunto de datos no tiene.

- **Modelos *Seq2Seq* con decodificador autoregresivo**: son más costosos de entrenar y de ejecutar en la aplicación, y no resuelven una limitación que CTC tenga en este problema, porque el texto sigue el mismo orden que el deletreo.

4.7 Configuración común

Para que la comparación aísle el efecto de la arquitectura, los tres modelos usan:

- **Las mismas entradas:** las 80 características por cuadro de la sección 4.3, y solo los cuadros que la máscara marca como válidos.
- **La misma partición por participante:** `train` actualiza los pesos, `val` elige hiperparámetros y checkpoints, y `test` se usa una sola vez al final.
- **El mismo entrenamiento:** optimizador AdamW, recorte del gradiente a norma 1, lotes de 32 secuencias con relleno dinámico y semilla 42.
- **La misma decodificación:** voraz, es decir, el símbolo más probable de cada cuadro seguido de la función *B* de CTC.
- **El mismo filtro:** se excluyen las secuencias que no cumplen la longitud mínima de CTC, el 3.8% del corpus.

4.8 Ajuste de hiperparámetros

El presupuesto de GPU disponible (Kaggle, una NVIDIA T4) no permite una búsqueda exhaustiva sobre 54,000 secuencias. Se usó una **búsqueda reducida y documentada**: cada configuración se entrenó 3 épocas sobre 5,000 secuencias de entrenamiento y se evaluó con la distancia de edición normalizada sobre el mismo conjunto de validación (cobertura ≥ 50%). Se exploraron tres ejes:

1. **Capacidad del codificador**, en los tres modelos: una versión base y una más grande (más unidades o canales y más capas).
2. **Tasa de aprendizaje** en M3 (10^{-3} frente a $5 \cdot 10^{-4}$). Los Transformers son más sensibles a ella que las redes recurrentes.
3. **Umbral de cobertura** en M1 (20% frente a 50%). Esta decisión quedó pendiente en la preparación de datos: con 20% se entrena con el 90% del corpus, pero con secuencias más incompletas.

La configuración con menor distancia de edición en validación de cada arquitectura se entrenó luego con todos los datos. El entrenamiento final se detiene en la primera de tres condiciones: 15 épocas, 3 épocas seguidas sin mejorar la pérdida de validación (*early stopping*) o 30 minutos de GPU por modelo. Siempre se conserva el checkpoint con menor pérdida de validación.

4.9 Evaluación

Cada modelo final se evaluó una vez sobre `test`, con 10 participantes que no intervinieron ni en el entrenamiento ni en la selección de hiperparámetros. Se reportan:

- **Distancia de edición normalizada** (métrica principal; sección 3.6).
- **CER**, y el puntaje de la competencia $1 - \text{CER}$.
- **Proporción de frases exactas**.
- **Error por tipo de carácter** (letras, dígitos, símbolos y espacios), para contrastar con el desbalance del vocabulario hallado en el EDA.
- **Error según la longitud de la frase**.
- **Costo:** número de parámetros, tiempo de inferencia y tiempo de entrenamiento.

5. Resultados y análisis

Conjunto de datos, limpieza y análisis exploratorio

Las cifras del corpus de frases se calcularon sobre las 67,208 secuencias de `train.csv`. Las de la señal provienen del análisis exploratorio de un archivo oficial completo (1,000 secuencias, 162,699 cuadros), salvo donde se indica que corresponden al corpus completo.

5.1 El corpus de frases

- **Tamaño.** 67,208 secuencias, 46,478 frases distintas y cerca de 1.2 millones de caracteres.
- **Longitud.** Las frases miden entre 1 y 31 caracteres, con media 17.8, mediana 17 y desviación estándar 5.7. La mitad central va de 12 a 22.
- **Contenido.** No es lenguaje común. Según una clasificación aproximada por reglas sobre el texto, cerca del 32 % de las frases son direcciones, el 28 % direcciones web y el 24 % números de teléfono; el 16 % restante es otro tipo de texto, como nombres.
- **Vocabulario.** Aparecen 59 caracteres. Las letras son el 61.2 % del texto, los dígitos el 24.6 %, los símbolos el 9.3 % y los espacios el 4.9 %. El desbalance está en la cola: 17 caracteres aparecen en menos de una de cada mil posiciones, y varios símbolos (`$`, `#`, `!`, `*`, `[`, `;`) casi no aparecen.
- **Participantes.** Cada participante aporta en mediana 795 secuencias; el que más, 1,535, y el que menos, 1. El coeficiente de Gini de los aportes es 0.16, es decir, ningún firmante domina el corpus.

El peso de los dígitos es una consecuencia directa del contenido: en este corpus los números no son una clase rara, como sí lo son los símbolos poco usados.

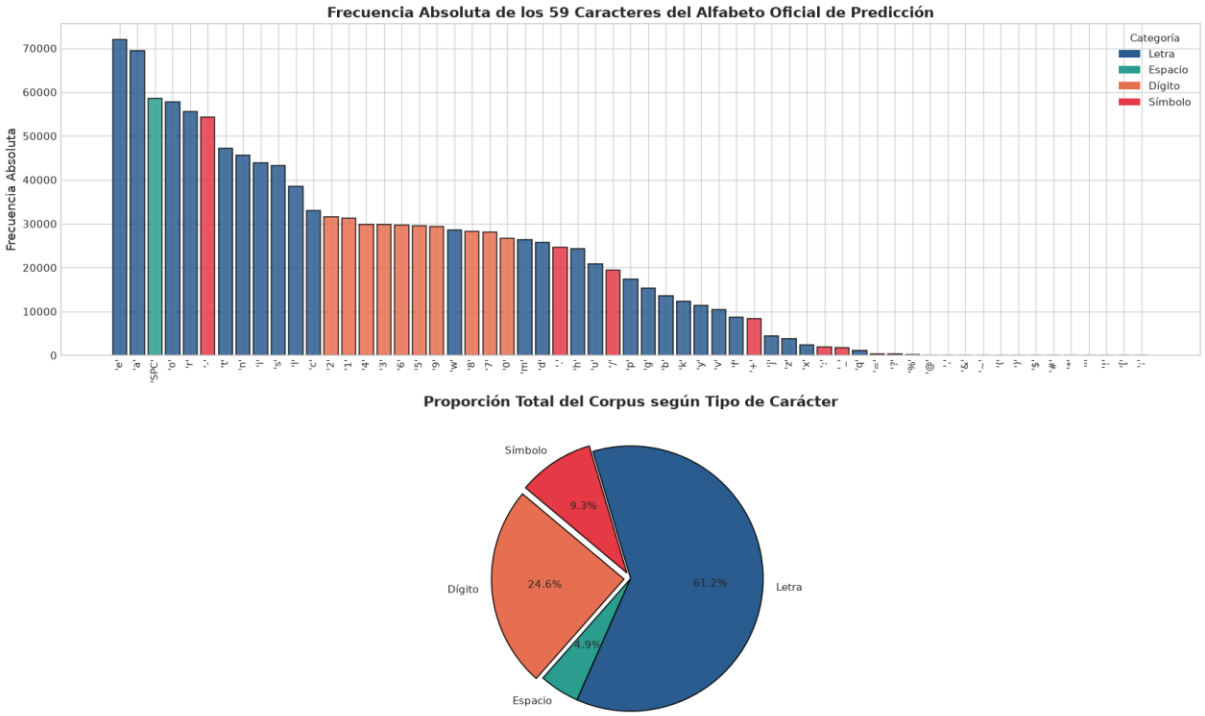


Figura 1. *Frecuencia de los 59 caracteres del vocabulario y proporción por tipo de carácter en el corpus de frases.*

5.2 Estructura y calidad de la señal

- **Composición.** De las 1,629 coordenadas por cuadro, 1,404 son de la cara (86.2 %), 99 del cuerpo (6.1 %) y 126 de las manos (7.7 %). Leer solo manos y cuerpo reduce la memoria al 13.8 %.
- **La ausencia es por bloque.** MediaPipe entrega los 21 puntos de una mano o ninguno; no se encontraron detecciones parciales. El cuerpo aparece en todos los cuadros.
- **Una sola mano trabaja.** La mano no dominante está ausente en más del 90 % de los cuadros en el 99.2 % de las secuencias. No es un defecto: es la huella de una actividad unimanual. La mano activa es la derecha en el 79.6 % de las secuencias y la izquierda en el 20.4 %.
- **La mano activa también se pierde.** En promedio falta en el 44 % de los cuadros de una secuencia, y en el 42.3 % de las secuencias falta en más de la mitad. El 41.6 % de los cuadros no tiene ninguna mano detectada. Ninguna secuencia carece por completo de ambas manos.
- **Los huecos son cortos pero pesan poco.** La mediana de un hueco es de 3 cuadros y el 67.9 % mide 5 o menos, pero esos huecos cortos solo suman el 22 % de los cuadros ausentes. Unos pocos huecos largos (hasta 324 cuadros) concentran el resto.

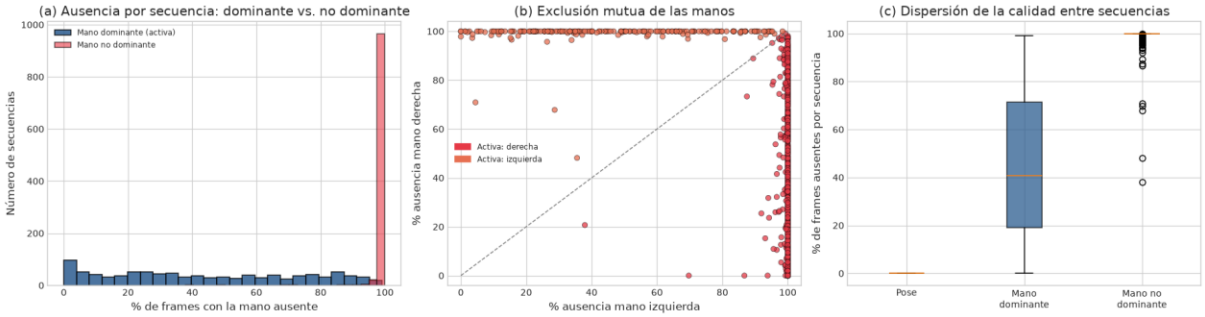


Figura 2. *(a) Ausencia de la mano activa y de la no dominante por secuencia; (b) las secuencias se ubican sobre los ejes, no en la diagonal: una mano trabaja y la otra no; (c) dispersión de la ausencia por canal.*

5.3 Qué estrategia de relleno usar

Para decidir cómo tratar los huecos se hizo un experimento con respuesta conocida: en tramos donde la mano sí estaba, se ocultaron huecos de 1 a 21 cuadros y se midió el error de cada estrategia al reconstruirlos (2,807 huecos en 401 secuencias).

Estrategia	Error absoluto medio	Veces peor que la mejor
Interpolación lineal	0.0142	1.0
Repetir el último valor	0.0209	1.5
Media de la secuencia	0.0304	2.1
Rellenar con ceros	0.3713	26.2

Rellenar con ceros es 26 veces peor que interpolar, porque en coordenadas normalizadas el cero es una esquina de la imagen y equivale a teletransportar la mano. El error de la interpolación crece con la longitud del hueco (de 0.007 con un cuadro a 0.025 con 21). De ahí salen las dos reglas del preprocesamiento: interpolar solo huecos de hasta 5 cuadros y marcar el resto como no válido en lugar de inventarlo.

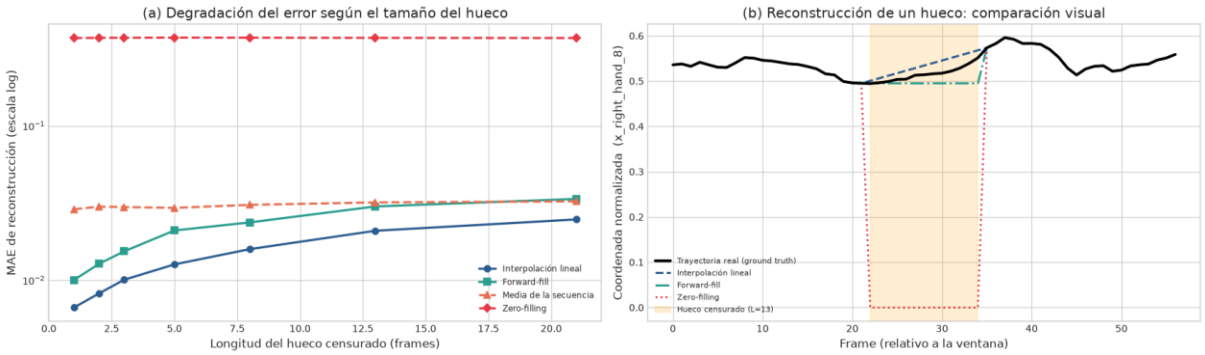


Figura 3. *(a) Error de reconstrucción de cada estrategia según la longitud del hueco (escala logarítmica); (b) ejemplo de un hueco de 13 cuadros reconstruido con cada estrategia.*

5.4 Artefactos de seguimiento

La velocidad mediana del centroide de la mano es de 0.009 unidades por cuadro y el percentil 99.9 es 0.109. El criterio estadístico por sí solo marcaría el 5.3 % de las transiciones, demasiadas para ser errores.

El criterio físico (más de 0.15) marca el 0.028 %, y todas coinciden con el estadístico. Exigir ambos deja 26 transiciones en 23 secuencias. Son pocos casos, y por eso se invalida el cuadro en vez de descartar la secuencia.

5.5 Dimensión temporal

- **Duración.** Las secuencias miden en mediana 148 cuadros, con un rango de 1 a 672 y asimetría positiva. Rellenar todas hasta la longitud máxima desperdiciaría la mayor parte del cómputo, así que los lotes se rellenan solo hasta la secuencia más larga de cada lote.
- **Relación con el texto.** La duración crece con la longitud de la frase: correlación de Pearson de 0.62 y una pendiente de 9.2 cuadros por carácter (unos 0.3 segundos a 30 cuadros por segundo).
- **El firmante importa.** El firmante más lento usa 3.5 veces más cuadros por carácter que el más rápido (16.0 contra 4.6). La prueba de Kruskal-Wallis rechaza que la velocidad sea igual entre firmantes ($H = 398.2$, 89 grados de libertad, $p < 0.001$), y el firmante explica el 38.2 % de la varianza. Este resultado es el que justifica partir los datos por participante.

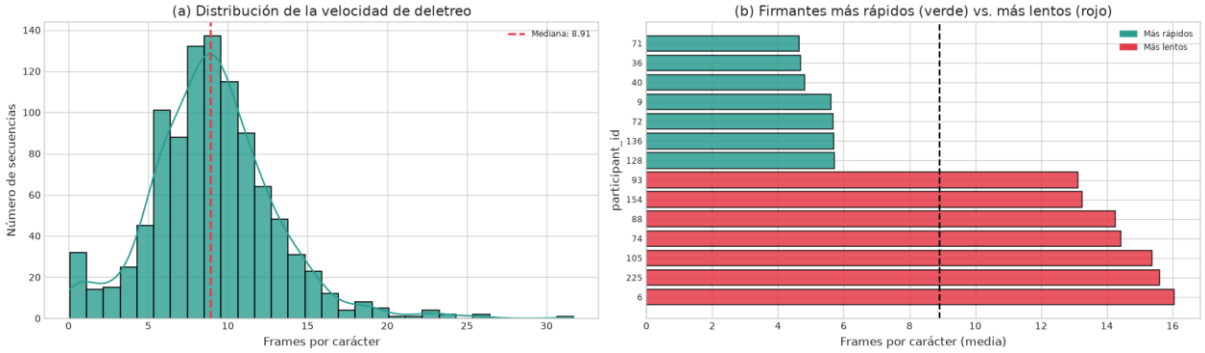


Figura 4. *(a) Distribución de cuadros por carácter; (b) firmantes más rápidos y más lentos.*

5.6 Resultado de la preparación sobre el corpus completo

Al ejecutar el preprocesamiento sobre los 68 archivos se obtuvo lo siguiente:

1. **La calidad depende del firmante.** La cobertura media de la mano activa por participante va de 33 % a 94 %. El conjunto de validación, con solo 9 participantes, quedó con una cobertura media de 59 %, menor que la de entrenamiento (69 %) y prueba (70 %). La validación es, por lo tanto, algo más difícil que la prueba, y hay que tenerlo presente al comparar sus métricas.
2. **El umbral de descarte tiene un costo alto.** Exigir 50 % de cobertura conserva el 71.1 % del corpus; exigir 20 % conserva el 90.5 %. Como el modelo recibe la máscara de validez, el umbral se trata como un hiperparámetro del entrenamiento.
3. **CTC es viable en casi todo el corpus.** La mediana es de 8.8 cuadros por carácter. Solo 2,527 secuencias (3.8 %) tienen menos cuadros que los que su frase exige y se excluyen del entrenamiento.

4. **Datos efectivos de entrenamiento.** Con cobertura mínima de 50 % y la condición de CTC, los modelos se entrenaron con 37,366 secuencias y se validaron con 3,857.
5. **Consistencia entre entrenamiento y aplicación.** Una secuencia cruda exportada para la aplicación produce exactamente las mismas características que las guardadas para entrenar. Se comprobó en local y en Kaggle.

5.7 Limitaciones

- El análisis de la señal de las secciones 5.2 a 5.5 se hizo sobre un archivo de los 68. Las cifras de la sección 5.6 sí corresponden al corpus completo y son coherentes con las del archivo (por ejemplo, 71.3 % de retención en el archivo y 71.1 % en el corpus).
- El conjunto de validación tiene 9 participantes y el de prueba 10. Con tan pocos firmantes, un solo participante de mala calidad mueve las métricas.
- Todo el sistema hereda los errores de MediaPipe: si el detector no encuentra la mano, no hay nada que el modelo pueda recuperar.
- La cara se descartó. En ASL la boca y la expresión aportan información, aunque su papel en el deletreo es secundario; no se midió cuánto se pierde.

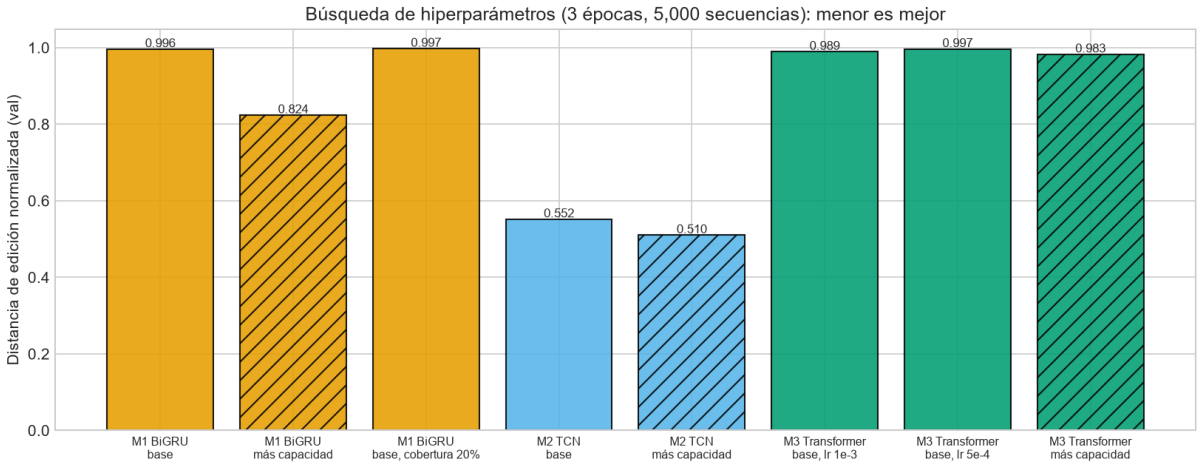
Modelos

Todos los modelos se entrenaron en Kaggle con una GPU NVIDIA T4, ejecutando el notebook maestro de principio a fin, y se evaluaron sobre 4,181 frases de prueba de 10 participantes no vistos.

5.8 Ajuste de hiperparámetros

Tabla 5.1. Búsqueda reducida: 3 épocas sobre 5,000 secuencias, evaluada en el mismo conjunto de validación (Figura 5).

Modelo	Configuración	Parámetros	Distancia de edición (val)
M1	Base (128 unidades, 2 capas)	473,405	0.996
M1	**Más capacidad (192 unidades, 3 capas)**	1,670,845	**0.824**
M1	Base, cobertura ≥ 20%	473,405	0.997
M2	Base (128 canales, 4 niveles)	347,453	0.552
M2	**Más capacidad (192 canales, 6 niveles)**	1,136,701	**0.510**
M3	Base, tasa 10^{-3}	416,189	0.989
M3	Base, tasa $5{\cdot}10^{-4}$	416,189	0.997
M3	**Más capacidad (d = 192, 4 capas), tasa $5{\cdot}10^{-4}$**	1,216,189	**0.983**

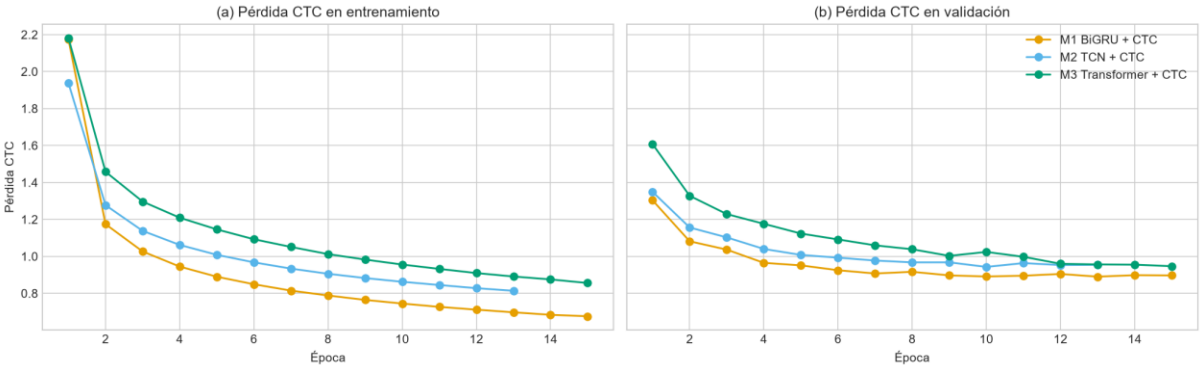


La búsqueda favoreció la mayor capacidad en las tres arquitecturas. Su principal limitación es el presupuesto: con 3 épocas, M1 y M3 apenas empezaban a emitir caracteres (distancia cercana a 1), mientras que la TCN, que converge más rápido, ya llegaba a 0.51. Por eso la búsqueda sirve para elegir dentro de cada arquitectura, pero no para compararlas entre sí. La comparación de cobertura (20% contra 50%) quedó sin decidir (0.997 contra 0.996), y se mantuvo el umbral de 50% del análisis exploratorio.

5.9 Entrenamiento final

Tabla 5.2. Entrenamiento final con la configuración elegida (37,366 secuencias; Figura 6).

	M1 BiGRU	M2 TCN	M3 Transformer
Épocas ejecutadas (mejor)	15 (13)	13 (10)	15 (15)
Segundos por época	83	53	97
Minutos de entrenamiento	20.8	11.4	24.3
Pérdida CTC en validación	0.890	0.943	0.947



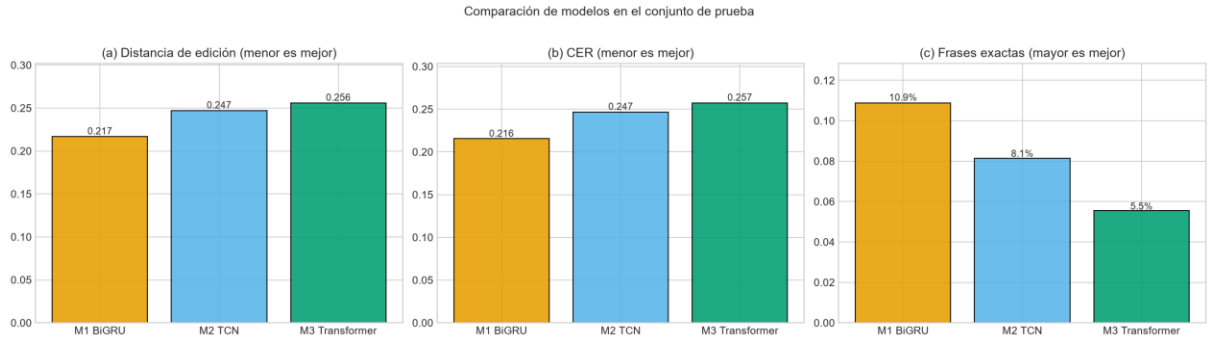
M2 se detuvo por *early stopping*: no mejoró en tres épocas seguidas. M1 llegó a su mínimo en la época 13 y se estabilizó. M3 obtuvo su mejor pérdida en la **última** época, señal de que todavía no había convergido. Ningún modelo alcanzó el límite de 30 minutos.

5.10 Comparación en el conjunto de prueba

Tabla 5.3. Resultados en prueba (Figura 7).

Modelo	Distancia de edición	CER	1 – CER	Frases exactas	Parámetros	ms por secuencia*
M1 BiGRU + CTC	**0.217**	**0.216**	**0.784**	**10.9%**	1.67 M	0.77
M2 TCN + CTC	0.247	0.247	0.753	8.1%	1.14 M	**0.35**
M3 Transformer + CTC	0.256	0.257	0.743	5.6%	1.22 M	0.71

* Inferencia en GPU T4, por lotes de 32 secuencias.

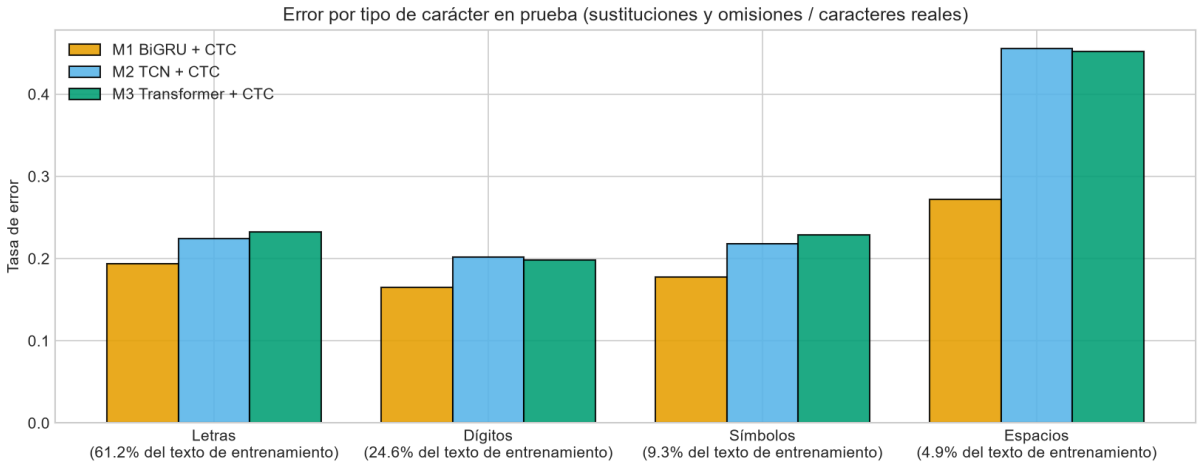


M1 es el mejor modelo en las tres métricas de calidad. Transcribe correctamente unos 78 de cada 100 caracteres, y más de una de cada diez frases sin ningún error. El orden de los modelos es el mismo en validación (0.205, 0.232 y 0.235). Las cifras de prueba son algo peores que las de validación en todos los modelos, a pesar de que, según la preparación de datos, la validación era el conjunto con menor cobertura. Lo más probable es que la diferencia se deba a la variabilidad entre firmantes: con 9 y 10 participantes por conjunto, unos pocos firmantes pesan mucho en el promedio.

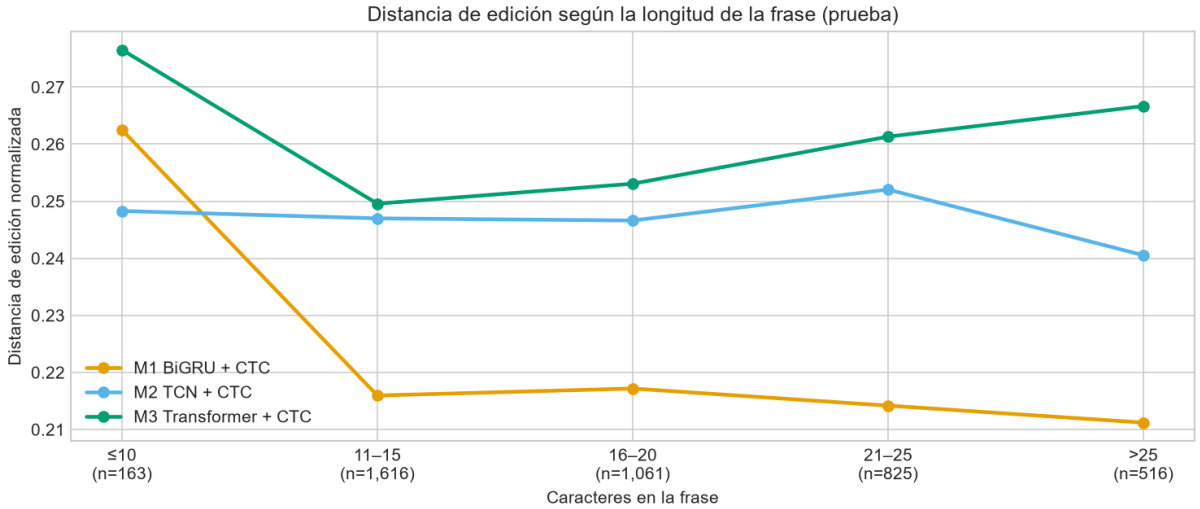
Respecto de la primera versión de M1 (128 unidades, 8 épocas y lotes de 16), que obtuvo 0.244 en validación, la configuración final mejora la distancia de edición en unos 4 puntos.

5.11 Análisis de errores

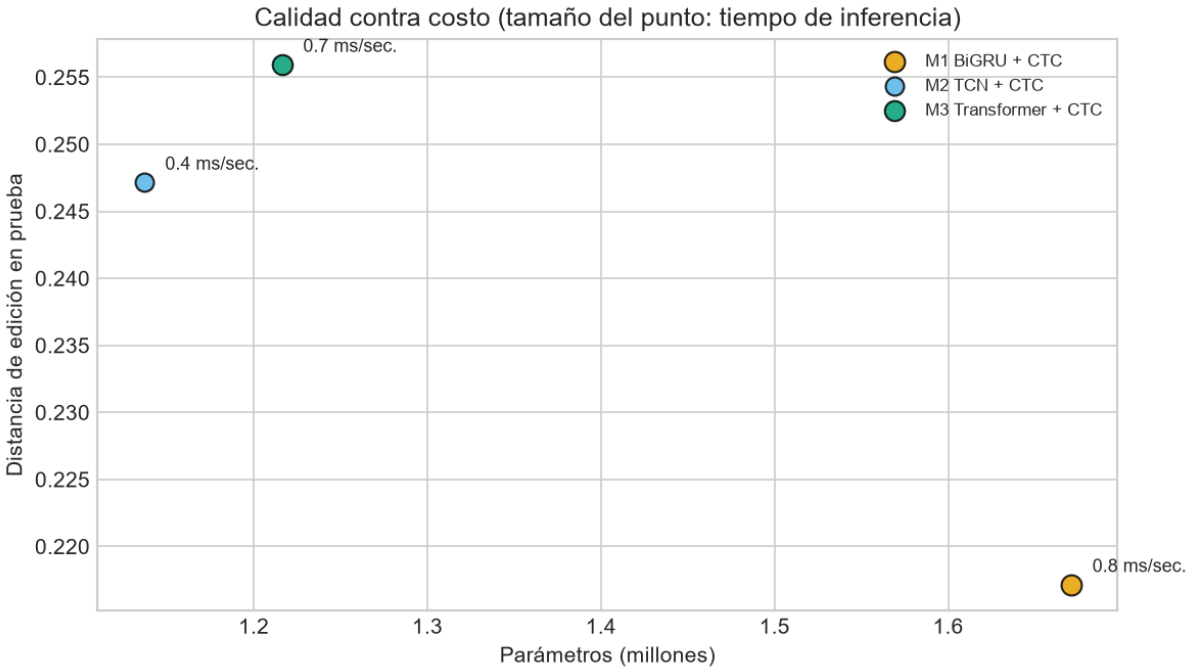
Por tipo de carácter (Figura 8). Las letras son el 61% del texto de entrenamiento y los dígitos el 25%. Aun así, las letras fallan más (19.4% en M1) que los dígitos (16.5%) y que los símbolos (17.8%; 9% del texto). La frecuencia, por sí sola, no explica el error. Los dígitos tienen configuraciones de mano muy distintas entre sí y aparecen en patrones repetitivos, como teléfonos y códigos postales. Las letras, en cambio, incluyen pares con formas parecidas que la coarticulación vuelve más ambiguos. **Los espacios son el punto más débil**: 27% de error en M1 y 45% en M2 y M3. No tienen seña propia, así que el modelo debe deducirlos de las pausas entre palabras. M1, que integra contexto en ambos sentidos de toda la frase, es el que mejor lo resuelve. Las ventanas locales de la TCN y el Transformer, sin suficiente entrenamiento, no lo logran.



Por longitud de la frase (Figura 9). El error de M1 es estable entre 11 y más de 25 caracteres (0.21–0.22). Solo las frases muy cortas (10 caracteres o menos; 163 casos) son algo peores (0.26), porque cada error pesa más en el promedio. CTC maneja bien las secuencias largas, como las URLs y las direcciones que forman la cola de la distribución vista en el EDA.



Calidad contra costo (Figura 10). M2 tiene la mitad del tiempo de inferencia y la mitad del tiempo de entrenamiento de M1, a cambio de 3 puntos de distancia de edición. En la aplicación, que corre en CPU, los tres modelos responden en milisegundos, así que la diferencia de costo no limita su uso.



Ejemplos. Los errores típicos son omisiones y sustituciones de caracteres sueltos, no frases inventadas. Por ejemplo, "271097 bayshore boulevard" se transcribe como "271097 bayshore borevard" (M1), y "+20-4703-88149" se transcribe perfectamente con los tres modelos. Menos del 0.1% de las frases quedan vacías.

5.12 Aplicación interactiva

Propósito y alcance

La aplicación permite explorar los datos y transcribir una secuencia de landmarks de MediaPipe a texto. La entrada conserva el formato crudo de Kaggle; la limpieza se ejecuta automáticamente con `src/preprocessing.py`, el mismo módulo utilizado para preparar los datos de entrenamiento. El sistema reconoce secuencias de deletreo de ASL. No traduce lenguaje de señas completo ni extrae landmarks desde videos o una cámara.

La navegación contiene tres páginas: **Explorar datos**, **Predecir** y **Rendimiento**. El orden de la demostración comienza con una predicción para mostrar el resultado concreto, continúa con los datos que la explican y termina con los errores y el costo computacional.

Tecnologías y organización

Se utilizó **Streamlit** para formularios, carga de archivos, navegación y controles de visibilidad. Su integración con Python permite reutilizar directamente los contratos de preprocesamiento y modelado. **Plotly** aporta gráficas con hover, leyendas, controles de animación y selección de frames. **Pandas**, **NumPy** y **PyArrow** permiten leer CSV y Parquet y transformar coordenadas. **PyTorch** carga los checkpoints en CPU y ejecuta el modelo sin conversión adicional de sus pesos.

Las vistas se encuentran en `app/views/`. La carga de datos, inferencia, gráficas y comparación de caracteres se separan en `app/lib/`. Los datos y modelos se resuelven con rutas relativas a la ubicación del repositorio. La configuración visual está en `.streamlit/config.toml`; las fuentes están incluidas en `app/static/fonts/`. Los recursos necesarios para la demostración son locales y no requieren acceso a Kaggle.

Los modelos cargados se conservan en caché de recursos. Los metadatos, muestras e historiales utilizan caché de datos. Al cambiar pesos o métricas se reinicia la aplicación para refrescar esas cachés. La primera inferencia puede ser más lenta por el calentamiento; por eso el benchmark realiza una llamada inicial por modelo antes de registrar tiempos.

Exploración de datos limpios

La vista inicia con las **30 muestras limpias** incluidas en el repositorio. Sus distribuciones de frases corresponden a 8 firmantes de test y no se extrapolan al corpus completo. Un control separado permite consultar los metadatos originales de `data/train.csv`: 67 208 frases de 94 firmantes. La partición completa contiene 75 firmantes de entrenamiento, 9 de validación y 10 de prueba, con semilla 42.

Se muestran frecuencia y categoría de caracteres, longitud de frases y aporte por participante. Para una secuencia elegida se puede consultar presencia de mano y pose, trayectoria normalizada de la punta del índice, esqueleto animado y distancia de los 21 landmarks a la muñeca en unidades de palma. La cobertura y su resumen por mano activa se calculan después de la limpieza. Las pérdidas de tracking se distinguen de los frames válidos; un salto de trayectoria no se interpreta automáticamente como un cambio de letra.

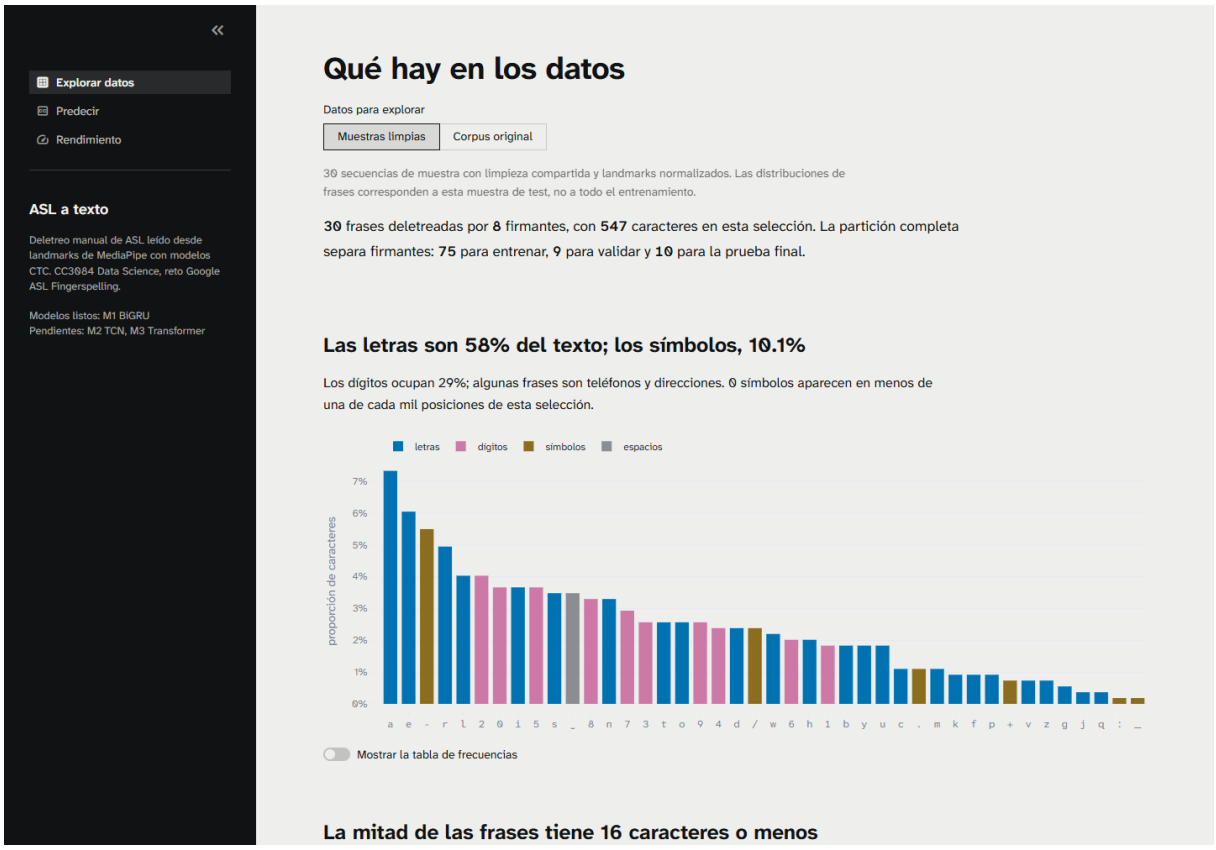


Figura 11. *Distribución de caracteres de las muestras limpias. El selector distingue esta vista del corpus original.*

Predicción y comparación

El usuario selecciona una muestra o carga un archivo CSV o Parquet. Si el archivo contiene varias secuencias, elige explícitamente su `sequence_id`. La entrada debe contener las 63 coordenadas de al menos una mano completa; la pose y la mano restante pueden faltar y se representan como ausentes. Si no existe `frame`, se conserva el orden de las filas y se genera un índice temporal.

El pipeline selecciona la mano activa, refleja las secuencias izquierdas, invalida picos de velocidad, recorta extremos ausentes, interpola huecos internos de hasta cinco frames y normaliza la mano y la pose. Produce **80 características por frame** y una máscara booleana. Solo los frames válidos se envían al codificador temporal. Un control opcional permite consultar el diagnóstico de los siete pasos sin exigir al usuario realizar la limpieza.

La selección de modelos admite uno, varios o todos los checkpoints disponibles. Para cada uno se muestra texto, tiempo de inferencia en CPU y, cuando hay una frase de referencia, distancia de edición y distancia normalizada. Las sustituciones, omisiones e inserciones se alinean carácter por carácter y se identifican mediante marcas y colores. Una tabla y su descarga CSV permiten conservar la comparación. La referencia ingresada no se modifica silenciosamente: espacios y mayúsculas afectan la distancia de edición.

La animación presenta la mano normalizada, la pista de emisiones CTC y el texto acumulado. La probabilidad de un token por frame no se presenta como una confianza calibrada de la frase completa. La

reproducción usa frames válidos y una cadencia de visualización de 30 fps; no reconstruye la duración original de los huecos descartados.

En la muestra de referencia `109-580-5359`, M1 produce `109-580-5355` (una sustitución), M2 omite el último dígito (`109-580-535`) y M3 la transcribe sin errores. Es un ejemplo individual y no reemplaza la evaluación agregada: en el conjunto de prueba completo, M1 es el mejor modelo.

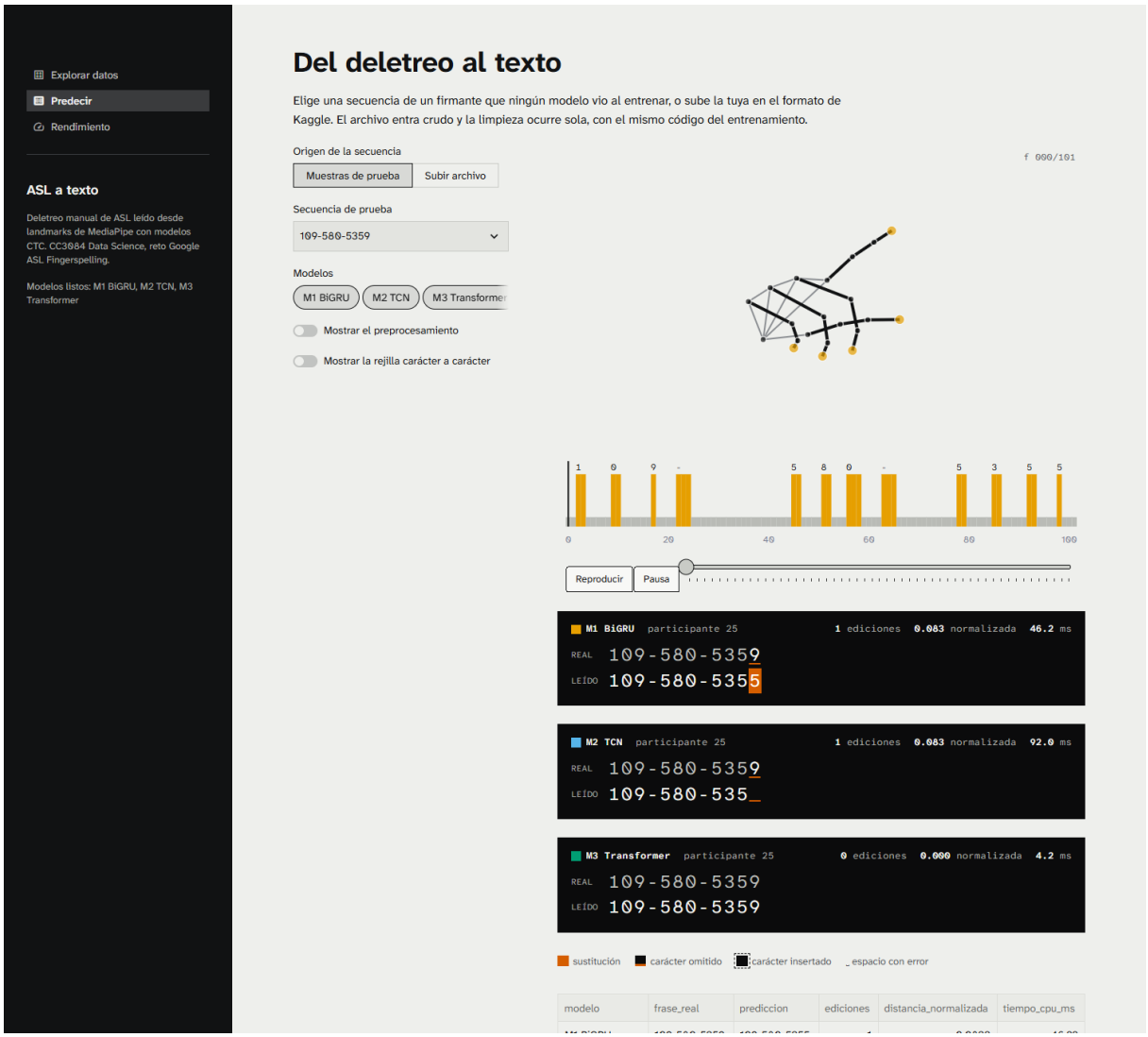


Figura 12. *Animación de la mano, emisiones CTC y subtítulos de los tres modelos con la frase real y la predicción alineadas carácter por carácter.*

Rendimiento e interpretación

Las cifras se leen de `models/metrics.json`. La página ofrece selección de modelos, tabla, descarga de métricas y controles para mostrar u ocultar métricas globales, errores por categoría, relación frecuencia-error y curvas de entrenamiento. Cuando existen resultados de validación y prueba o tamaños distintos, se presentan en grupos separados. La coincidencia en split y número de secuencias permite agrupar

entradas, pero debe verificarse en el notebook que compartan el mismo conjunto evaluado antes de realizar una comparación científica.

La aplicación muestra los tres modelos evaluados en el conjunto de **prueba** (4 181 frases de 10 participantes no vistos). M1 BiGRU obtiene distancia de edición normalizada **0,217**, CER **0,216** y **10,9 %** de frases exactas; M2 TCN, 0,247, 0,247 y 8,1 %; M3 Transformer, 0,256, 0,257 y 5,6 %. La distancia normalizada es un promedio por frase; CER pondera los errores por el total de caracteres. El protocolo y el análisis completo están en `04_informe_modelos.md`.

El error por tipo de carácter de M1 es 19,4 % en letras, 16,5 % en dígitos, 17,8 % en símbolos y 27,2 % en espacios (M2 y M3 llegan a 45 % en espacios). Esta tasa cuenta omisiones y sustituciones sobre caracteres de referencia; las inserciones se registran aparte y sí intervienen en CER. Los espacios tienen el mayor error, pero el orden de frecuencias no explica por sí solo los resultados: las letras son el tipo más frecuente y fallan más que los dígitos y los símbolos. La asociación entre frecuencia y error no demuestra causalidad.

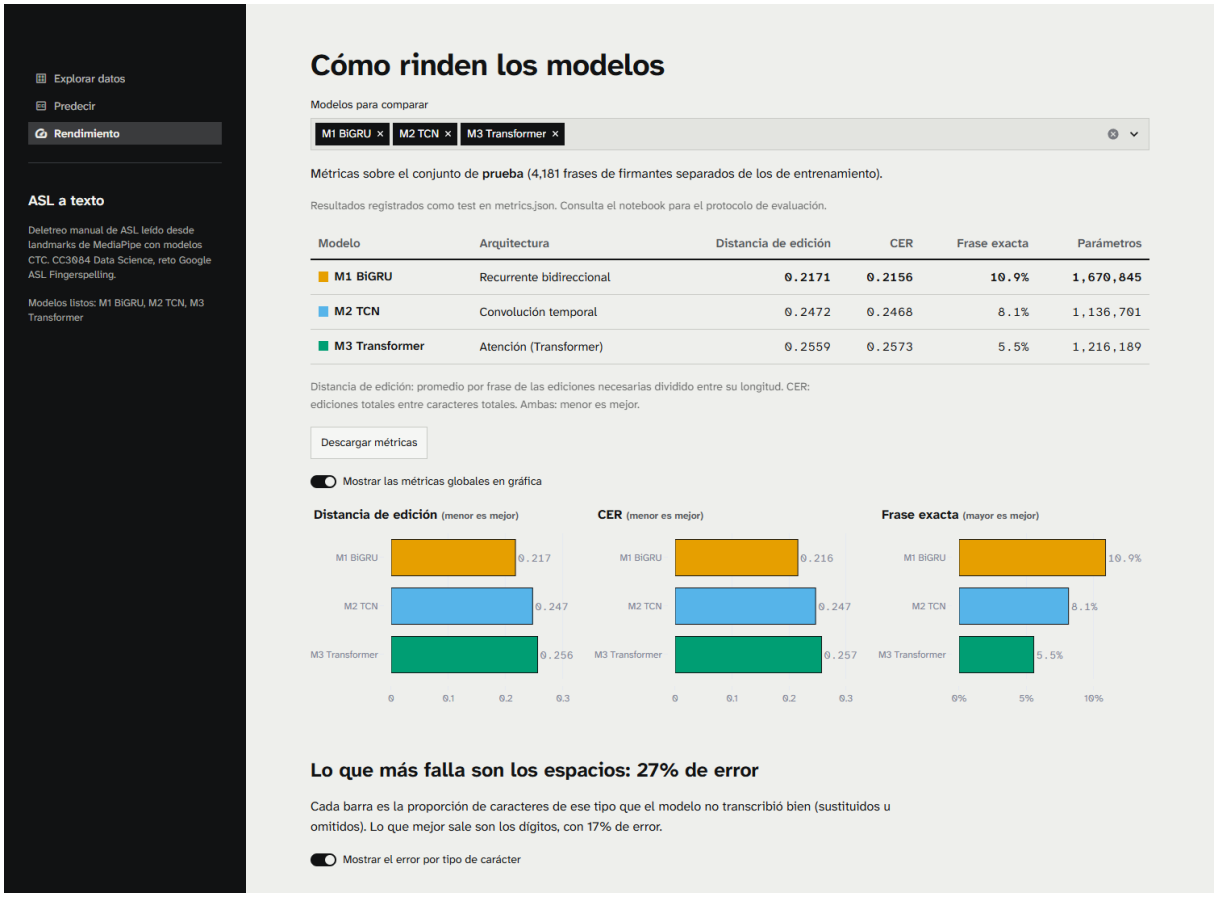


Figura 13. *Página de rendimiento con la comparación de los tres modelos en el conjunto de prueba y los controles para mostrar u ocultar cada gráfica.*

El botón **Medir en esta máquina** realiza una llamada de calentamiento por modelo y mide inferencia y decodificación sobre las 30 muestras. Excluye carga de pesos, lectura y preprocesamiento; no modifica las métricas guardadas ni vuelve a evaluar test. El diagrama combina CER, latencia media y parámetros. La demostración grabada registró aproximadamente **15,2 ms por secuencia** para M1; este valor depende

del equipo y de su carga. Los tiempos registrados durante la evaluación por lotes en GPU (0,35 a 0,77 ms por secuencia, sección 5.10) provienen de otro entorno y no son directamente comparables con la inferencia individual en CPU.

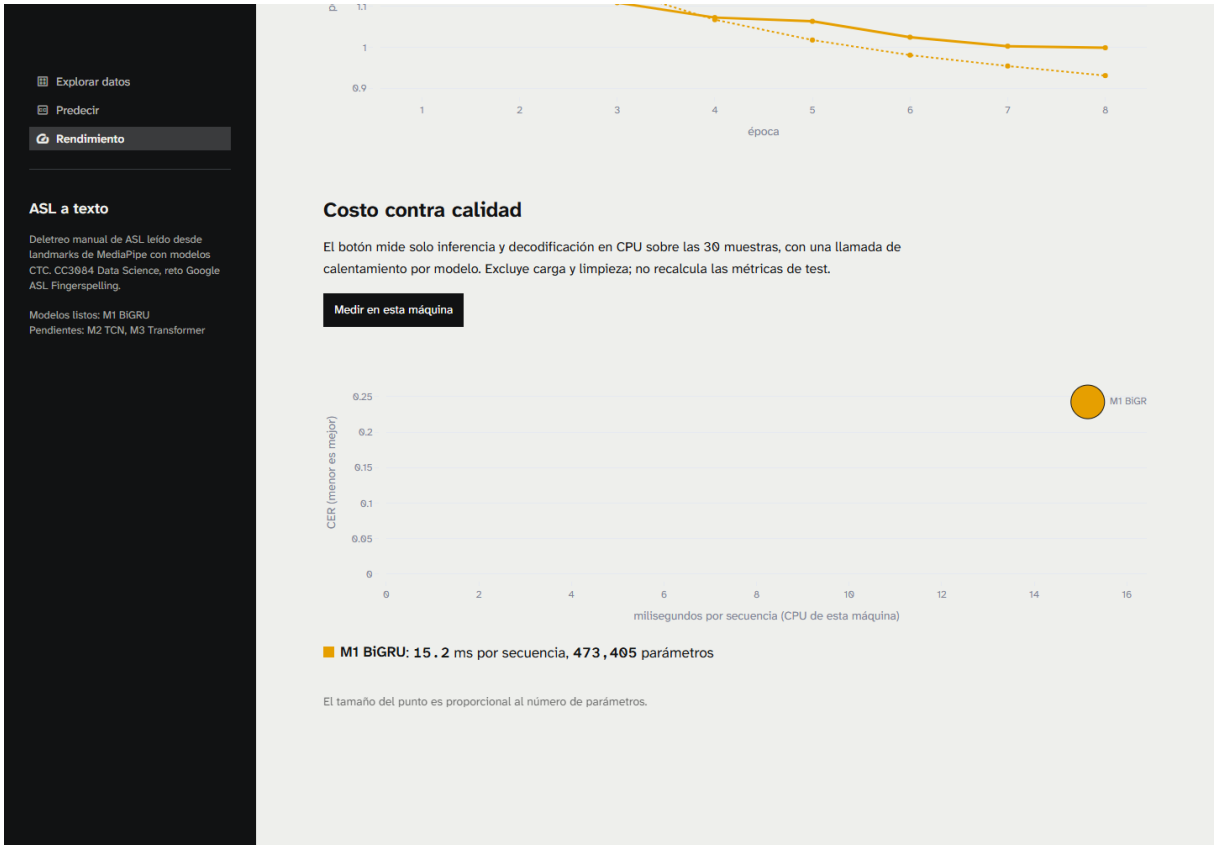


Figura 14. *Medición de eficiencia en CPU, con el promedio de latencia y el número de parámetros de M1.*

Diseño visual y accesibilidad

La interfaz utiliza una base acromática para reducir competencia visual con los resultados. El texto `#111213` sobre el fondo `#EEEFEC` tiene una relación de contraste aproximada de **16,25:1**; el texto secundario `#55585C`, **6,20:1**. Los subtítulos usan fondo `#0B0B0C` y texto `#F5F5F0`, con **17,99:1**. Estos valores se calcularon mediante luminancia relativa sRGB y se interpretan con el criterio de contraste de WCAG 2.2 (W3C, 2023).

Para identificar modelos se utiliza una paleta categórica basada en Okabe e Ito (2008): naranja para M1, azul para M2 y verde para M3. Cada modelo conserva su color entre páginas. Los nombres, contornos y marcas de error complementan el tono, de acuerdo con el principio de no depender exclusivamente del color. La magnitud de las coordenadas usa una escala secuencial azul con luminancia decreciente; la presencia anatómica utiliza una escala binaria acromática.

Las fuentes Atkinson Hyperlegible Next y Mono, desarrolladas para mejorar la diferenciación de caracteres, se incluyen localmente (Braille Institute, s. f.). La tipografía monoespaciada facilita la alineación de números y subtítulos. La justificación detallada y los códigos de color se conservan en

`reports/app_paleta.md`. Estos criterios de diseño no constituyen una certificación integral de accesibilidad.

Validación, errores y demostración

La carga rechaza archivos ilegibles, formatos ajenos a CSV o Parquet, entradas vacías, manos incompletas, coordenadas no numéricas o infinitas, frames no finitos, frames duplicados y registros sin identificador en archivos con `sequence_id`. Se limita la entrada a 10 000 frames. La ausencia total de una mano válida detiene la predicción con un mensaje comprensible; pocos frames generan una advertencia. Un fallo de inferencia se informa por modelo y permite conservar resultados de los demás.

Las pruebas cubren lectura CSV/Parquet, selección de múltiples secuencias, entradas inválidas, pose ausente, mano ausente, equivalencia entre la inferencia de la app y `src.models.predict`, renderizado de las tres vistas y controles de visibilidad. La demostración en navegador recorre predicción, exploración, rendimiento y benchmark con el checkpoint real.

La presentación está en `reports/presentacion/ASL_resultados.pptx`; el guion y las instrucciones de ensayo en `reports/guion_demo.md`. El video `reports/demo_app.mp4` permite mostrar el mismo recorrido si falla la demostración en vivo. Está grabado sin audio para acompañarlo con la explicación del expositor.

Integración de artefactos finales

La app carga BiGRU, TCN y Transformer mediante `src.models.load_model`. El repositorio incluye los checkpoints finales de los tres modelos, entrenados en Kaggle, y sus métricas de prueba en `metrics.json`. Las pruebas automáticas verifican que la inferencia de la app coincide con `src.models.predict` para cada arquitectura.

6. Conclusiones

6.1 Cumplimiento de los objetivos

Objetivo general. Se cumplió. Se construyó un sistema que transcribe a texto secuencias de *fingerspelling* representadas como landmarks. Comprende la preparación del corpus completo, tres modelos CTC comparados en firmantes no vistos y una aplicación interactiva. Todo se reproduce desde un único notebook ejecutado en Kaggle.

1. **Preprocesamiento único y reproducible.** Se cumplió. `preprocess_sequence` aplica los siete pasos derivados del análisis exploratorio y procesó las 67,208 secuencias (190 GB) en unos 9 minutos, reduciéndolas a 1.6 GB. Se verificó, en local y en Kaggle, que una secuencia cruda produce exactamente las mismas características al entrenar y en la aplicación.
2. **Partición por firmante.** Se cumplió. Los 94 participantes se repartieron en 75, 9 y 10, sin ningún firmante compartido entre conjuntos (el código lo comprueba antes de entrenar). Las métricas de prueba corresponden, por lo tanto, a personas que ningún modelo vio.
3. **Tres modelos CTC sobre el corpus completo.** Se cumplió. M1 (BiGRU), M2 (TCN) y M3 (Transformer) se ajustaron con una búsqueda de hiperparámetros y se entrenaron con 37,366 secuencias de los 68 archivos en una GPU T4, en 11 a 24 minutos cada uno.
4. **Comparación de los modelos.** Se cumplió. **M1 es el mejor modelo**: en prueba alcanza una distancia de edición de 0.217, un CER de 0.216 (puntaje de la competencia 0.784) y un 10.9% de frases exactas. **M2 es la alternativa eficiente**: la mitad del tiempo de entrenamiento y de inferencia, con 3 puntos más de error. **M3 obtuvo el peor resultado** con este presupuesto, pero no había convergido.
5. **Relación con el análisis exploratorio.** Se cumplió. El desbalance de caracteres no basta para predecir dónde fallan los modelos: las letras, el tipo más frecuente, fallan más que dígitos y símbolos, y el principal problema son los espacios, que no tienen seña propia. La decisión del EDA de no rellenar con ceros, sino interpolar huecos cortos y enmascarar el resto, permitió entrenar con el 71% del corpus. La partición por firmante, motivada por el 38% de varianza que explica el firmante, evitó una evaluación optimista.
6. **Aplicación.** Se cumplió. La aplicación recibe secuencias crudas en CSV o Parquet, las preprocesa sin intervención del usuario y muestra la transcripción de uno, varios o los tres modelos, junto con su rendimiento en prueba en gráficas interactivas que se pueden ocultar.

6.2 Efectividad de los algoritmos

La formulación como reconocimiento de secuencias con CTC fue la adecuada. Permitió entrenar sin saber dónde empieza cada letra, el error no crece con la longitud de la frase y la restricción de longitud mínima solo excluyó el 3.8% del corpus. Con el presupuesto disponible, la recurrencia bidireccional aprovechó mejor un conjunto de este tamaño que la atención, y es el modelo recomendado para la aplicación. Una distancia de edición de 0.217 significa que, en promedio, uno de cada cinco caracteres requiere corrección. Es útil para demostrar el enfoque, pero todavía insuficiente para un uso cotidiano sin revisión.

6.3 Limitaciones y trabajo futuro

El presupuesto de GPU obligó a una búsqueda de hiperparámetros reducida, que no permitió decidir el umbral de cobertura ni comparar las arquitecturas en igualdad de convergencia. Tampoco se usaron aumento de datos, decodificación con un modelo de lenguaje ni los datos suplementarios de la competencia. Son las tres mejoras con más potencial, en ese orden. M3, en particular, debería reentrenarse con más épocas y con un bloque convolucional (Conformer), como en las soluciones ganadoras de la competencia.

7. Referencias

Bai, S., Kolter, J. Z., y Koltun, V. (2018). *An empirical evaluation of generic convolutional and recurrent networks for sequence modeling*. arXiv. https://arxiv.org/abs/1803.01271

Bragg, D., Koller, O., Bellard, M., Berke, L., Boudreault, P., Braffort, A., Caselli, N., Huenerfauth, M., Kacorri, H., Verhoef, T., Vogler, C., y Ringel Morris, M. (2019). Sign language recognition, generation, and translation: An interdisciplinary perspective. En *Proceedings of the 21st International ACM SIGACCESS Conference on Computers and Accessibility* (pp. 16–31). ACM. https://doi.org/10.1145/3308561.3353774

Braille Institute. (s. f.). *Atkinson Hyperlegible Font*. https://www.brailleinstitute.org/freefont/

Camgoz, N. C., Hadfield, S., Koller, O., Ney, H., y Bowden, R. (2018). Neural sign language translation. En *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 7784–7793). IEEE. https://doi.org/10.1109/CVPR.2018.00812

Camgoz, N. C., Koller, O., Hadfield, S., y Bowden, R. (2020). Sign language transformers: Joint end-to-end sign language recognition and translation. En *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 10020–10030). IEEE. https://doi.org/10.1109/CVPR42600.2020.01004

Cao, Z., Hidalgo, G., Simon, T., Wei, S.-E., y Sheikh, Y. (2021). OpenPose: Realtime multi-person 2D pose estimation using part affinity fields. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 43*(1), 172–186. https://doi.org/10.1109/TPAMI.2019.2929257

Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., y Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. En *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1724–1734). Association for Computational Linguistics. https://doi.org/10.3115/v1/D14-1179

Chow, A., Cameron, G., Georg, M., Sherwood, M., Culliton, P., Sepah, S., Dane, S., y Starner, T. (2023). *Google – American Sign Language fingerspelling recognition* [Competencia]. Kaggle. https://kaggle.com/competitions/asl-fingerspelling

Georg, M., Tanzer, G., Hassan, S., Shengelia, M., Uboweja, E., Sepah, S., Forbes, S., y Starner, T. (2024). *FSboard: Over 3 million characters of ASL fingerspelling collected via smartphones*. arXiv. https://arxiv.org/abs/2407.15806

Graves, A., Fernández, S., Gomez, F., y Schmidhuber, J. (2006). Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. En *Proceedings of the 23rd International Conference on Machine Learning* (pp. 369–376). ACM. https://doi.org/10.1145/1143844.1143891

Graves, A., Mohamed, A., y Hinton, G. (2013). Speech recognition with deep recurrent neural networks. En *2013 IEEE International Conference on Acoustics, Speech and Signal Processing* (pp. 6645–6649). IEEE. https://doi.org/10.1109/ICASSP.2013.6638947

Gulati, A., Qin, J., Chiu, C.-C., Parmar, N., Zhang, Y., Yu, J., Han, W., Wang, S., Zhang, Z., Wu, Y., y Pang, R. (2020). Conformer: Convolution-augmented transformer for speech recognition. En *Interspeech 2020* (pp. 5036–5040). ISCA. https://doi.org/10.21437/Interspeech.2020-3015

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hochreiter, S., y Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Iglewicz, B., y Hoaglin, D. C. (1993). *How to detect and handle outliers*. ASQC Quality Press.

Kim, S., Gholami, A., Shaw, A., Lee, N., Mangalam, K., Malik, J., Mahoney, M. W., y Keutzer, K. (2022). Squeezeformer: An efficient transformer for automatic speech recognition. En *Advances in Neural Information Processing Systems* (Vol. 35). Curran Associates. https://arxiv.org/abs/2206.00888

Koller, O., Forster, J., y Ney, H. (2015). Continuous sign language recognition: Towards large vocabulary statistical recognition systems handling multiple signers. *Computer Vision and Image Understanding, 141*, 108–125. https://doi.org/10.1016/j.cviu.2015.09.013

Lea, C., Flynn, M. D., Vidal, R., Reiter, A., y Hager, G. D. (2017). Temporal convolutional networks for action segmentation and detection. En *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 1003–1012). IEEE. https://doi.org/10.1109/CVPR.2017.113

LeCun, Y., Bengio, Y., y Hinton, G. (2015). Deep learning. *Nature, 521*(7553), 436–444. https://doi.org/10.1038/nature14539

Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady, 10*(8), 707–710.

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., y Grundmann, M. (2019). *MediaPipe: A framework for building perception pipelines*. arXiv. https://arxiv.org/abs/1906.08172

Okabe, M., & Ito, K. (2008). *Color Universal Design (CUD): How to make figures and presentations that are friendly to colorblind people*. https://jfly.uni-koeln.de/color/

Organización Mundial de la Salud. (2021). *World report on hearing*. https://www.who.int/publications/i/item/9789240020481

Padden, C., y Gunsauls, D. C. (2003). How the alphabet came to be used in a sign language. *Sign Language Studies, 4*(1), 10–33. https://doi.org/10.1353/sls.2003.0026

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. En *Advances in Neural Information Processing Systems* (Vol. 32). Curran Associates. https://arxiv.org/abs/1912.01703

Rastgoo, R., Kiani, K., y Escalera, S. (2021). Sign language recognition: A deep survey. *Expert Systems with Applications, 164*, Artículo 113794. https://doi.org/10.1016/j.eswa.2020.113794

Schuster, M., y Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing, 45*(11), 2673–2681. https://doi.org/10.1109/78.650093

Shi, B., Brentari, D., Shakhnarovich, G., y Livescu, K. (2021). Fingerspelling detection in American Sign Language. En *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 4164–4173). IEEE. https://doi.org/10.1109/CVPR46437.2021.00415

Shi, B., Del Rio, A. M., Keane, J., Brentari, D., Shakhnarovich, G., y Livescu, K. (2019). Fingerspelling recognition in the wild with iterative visual attention. En *2019 IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 5399–5408). IEEE. https://doi.org/10.1109/ICCV.2019.00550

Shi, B., Del Rio, A. M., Keane, J., Michaux, J., Brentari, D., Shakhnarovich, G., y Livescu, K. (2018). American Sign Language fingerspelling recognition in the wild. En *2018 IEEE Spoken Language Technology Workshop (SLT)* (pp. 145–152). IEEE. https://doi.org/10.1109/SLT.2018.8639639

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., y Polosukhin, I. (2017). Attention is all you need. En *Advances in Neural Information Processing Systems* (Vol. 30). Curran Associates. https://arxiv.org/abs/1706.03762

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/

Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., y Grundmann, M. (2020). *MediaPipe Hands: On-device real-time hand tracking*. arXiv. https://arxiv.org/abs/2006.10214

Anexo: código y artefactos

- **Repositorio:** https://github.com/jcdiegolopez/DS-PR02
- **Pipeline completo (preparación, modelos y evaluación):** `notebooks/Proyecto2_Resultados_Master.ipynb`
- **Análisis exploratorio (fase 1):** `notebooks/Proyecto2_EDA_ASL_Master.ipynb`
- **Preprocesamiento compartido con la aplicación:** `src/preprocessing.py`
- **Modelos, entrenamiento y evaluación:** `src/models.py`, `src/train.py`, `src/evaluate.py`
- **Aplicación:** `app/app.py` (ejecutar con `streamlit run app/app.py`)
- **Pesos y métricas finales:** `models/`